

ASSIGNMENT – 5

Simulation of Consistent Hashing over MongoDB (Spring Boot)

AIM

To implement consistent hashing using Spring Boot and MongoDB, distributing Student records across three storage nodes (Docker containers) via a hash ring with virtual nodes. The experiment demonstrates that adding or removing a node only re-routes a small fraction of keys, unlike modulo hashing which reshuffles all records.

CODE

```
package com.ssn.hashing.hashing;

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.List;
import java.util.SortedMap;
import java.util.TreeMap;
import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.Set;

/**
 * A plain, easy-to-explain implementation of Consistent Hashing.
 *
 * How it works (this is what you explain to faculty):
 * 1. Imagine a circle (the "hash ring") with positions 0 .. 2^32-1.
 * 2. Every storage node is hashed to a few positions on this circle
 *    ("virtual nodes"). More virtual nodes = more even data spread.
 * 3. Every record's key (rollNo) is also hashed to a position on the circle.
 * 4. The record is stored on the FIRST node found by moving clockwise
 *    from the record's position. That's it - that's the whole algorithm.
 *
 * Because we use a TreeMap (sorted map), "moving clockwise" is just
 * "find the next key greater than or equal to mine", which TreeMap
 * gives us directly via tailMap().
 */
public class ConsistentHashRing {

    // Sorted map: position on the ring -> physical node name
    private final SortedMap<Long, String> ring = new TreeMap<>();

    // How many virtual points each physical node gets on the ring
```

```
private final int virtualNodesPerNode;

public ConsistentHashRing(int virtualNodesPerNode) {
    this.virtualNodesPerNode = virtualNodesPerNode;
}

/** Adds a physical node to the ring by placing its virtual nodes. */
public synchronized void addNode(String nodeName) {
    for (int i = 0; i < virtualNodesPerNode; i++) {
        long position = hash(nodeName + "#VN" + i);
        ring.put(position, nodeName);
    }
}

/** Removes a physical node and all of its virtual nodes from the ring. */
public synchronized void removeNode(String nodeName) {
    ring.entrySet().removeIf(entry -> entry.getValue().equals(nodeName));
}

// Returns the node responsible for storing/looking up this key.
public synchronized String getNode(String key) {
    if (ring.isEmpty()) {
        return null;
    }
    long position = hash(key);

    // tailMap gives everything clockwise from our position onwards.
    SortedMap<Long, String> tail = ring.tailMap(position);

    // If nothing is clockwise, wrap around to the first node on the ring.
    Long targetKey = tail.isEmpty() ? ring.firstKey() : tail.firstKey();
    return ring.get(targetKey);
}

/** Lists the distinct physical node names currently on the ring. */
public synchronized List<String> getNodes() {
    Set<String> distinct = new LinkedHashSet<>(ring.values());
    return new ArrayList<>(distinct);
}

/** Hashes any string into a long position on the ring, using MD5. */
private long hash(String key) {
    try {
        MessageDigest md5 = MessageDigest.getInstance("MD5");
        byte[] digest = md5.digest(key.getBytes());
    }
}
```

```
// Use the first 4 bytes of the MD5 digest as our position.
long position = ((long) (digest[0] & 0xFF) << 24)
                | ((long) (digest[1] & 0xFF) << 16)
                | ((long) (digest[2] & 0xFF) << 8)
                | ((long) (digest[3] & 0xFF));
return position;
} catch (NoSuchAlgorithmException e) {
    throw new RuntimeException("MD5 algorithm not available", e);
}
}
```

TASKS PERFORMED

Task	Description	Result
Implement hash ring	Built ConsistentHashRing using MD5 hash and TreeMap for sorted positions	Ring operational with 3 nodes and 3 virtual nodes each
Start MongoDB nodes	Docker Compose launched mongo-node1 (27017), node2 (27018), node3 (27019)	Three independent MongoDB instances running
Add student records	POST /students with rollNo, name, department, year	Records distributed across nodes per hash ring mapping
Check distribution	GET /students/distribution	Initial distribution e.g. node1: 5, node2: 4, node3: 6
Add 4th node	POST /nodes/add with new Mongo container	Only a small fraction of records moved; rest untouched
Remove a node	POST /nodes/remove	That node's records relocated; other nodes' data untouched

ANALYSIS

Consistent hashing maps both keys and nodes onto a circular hash ring (0 to $2^{32}-1$ using MD5). Each physical node gets virtual nodes (default 3) for even distribution. Student records (identified by rollNo) are assigned to the first node clockwise from their hash position. Adding a node only claims the slice between it and its clockwise predecessor — all other keys stay put. Removing a node only reassigns its own slice. This contrasts with modulo hashing ($\text{hash}(\text{key}) \% N$), where changing N reshuffles every key's bucket. The experiment confirms the core benefit: minimal data movement during scale-out/scale-in.

LEARNING OUTCOMES

- Implemented consistent hashing from scratch using MD5 and TreeMap
- Understood the role of virtual nodes in evenly distributing load
- Observed that adding/removing nodes affects only a small fraction of records
- Compared consistent hashing vs modulo hashing in terms of data movement
- Built a Spring Boot app with custom MongoTemplate management per node